

这是一个非常敏锐且触及大模型底层原理的好问题！这也是推测解码 (Speculative Decoding) 能够大幅提速的最核心密码。

要理解为什么这 4 个 Draft Token 可以被大模型并行 (同时) 处理, 我们需要回到 **Transformer** 架构的本源特性, 并区分大模型的两种工作模式: “写考卷”模式 (**Autoregressive Decode**) 与 “批考卷”模式 (**Prefill / Forward Pass**)。

1. 小模型的“写考卷”模式: 必须串行

如图 1 (草稿阶段) 所示, 小模型在生成这 4 个字时, 是纯粹的自回归 (Autoregressive)。

它必须先猜出“我”, 然后把“我”放进去猜出“爱”, 再把“我爱”放进去猜出“中”.....它不知道未来是什么, 前一步的输出是后一步的输入, 所以绝对无法并行, 只能一步一步串行生成。

2. 大模型的“批考卷”模式: 天生并行

如图 2 (验证阶段) 所示, 当小模型把 [我, 爱, 中, 国] 这 4 个字交给大模型时, 大模型已经提前知道了这 4 个输入。

这使得大模型可以直接进入类似处理初始 Prompt 时的 **Prefill** (预填充) 模式。在这个模式下, Transformer 的自注意力机制 (**Self-Attention**) 天然支持极度并行的矩阵计算。

大模型在一次前向传播 (单次计算) 中, 同时做出了 4 个位置的预测:

- 位置 1: 大模型看到上下文 [Prompt], 算出一个预测概率分布。(用来验证小模型猜的第 1 个字“我”对不对)
- 位置 2: 大模型看到上下文 [Prompt, 我], 算出一个预测概率分布。(用来验证小模型猜的第 2 个字“爱”对不对)
- 位置 3: 大模型看到上下文 [Prompt, 我, 爱], 算出一个预测概率分布。(用来验证小模型猜的第 3 个字“中”对不对)
- 位置 4: 大模型看到上下文 [Prompt, 我, 爱, 中], 算出一个预测概率分布。(用来验证小模型猜的第 4 个字“国”对不对)

因为输入的字都已经摆在那里了, GPU 不需要等位置 1 算完再算位置 2。GPU 把这 4 个字的特征拼成一个大矩阵, 一个巨大的矩阵乘法直接就把 4 个位置的 **Q**、**K**、**V** 和最终的输出概率全部算出来了。

3. GPU 算力的极致压榨

我们在前面聊过“算力与显存”。在单字生成的 Decode 阶段, GPU 其实是“吃不饱”的, 因为单字的矩阵乘法太小了 (Memory-Bound, 显存带宽卡脖子)。

但是, 把 4 个 Draft Token 拼成一个矩阵交给大模型并行计算, 刚好发挥了 GPU 最擅长的事情——大规模稠密矩阵乘法。大模型并行验证 4 个字所花的时间, 和它单独只生成 1 个字所花的时间, 几乎是一模一样的!

 **总结: 推测解码的暴力美学**

- 小模型花了一点点时间, 串行蒙了 4 个字。
- 大模型利用其天然的并行计算能力, 在原本只够生成 1 个字的时间里, 一次性“批改”了这 4 个字的对错, 并顺手算出了第 5 个字。

如果小模型全猜对了, 大模型在这个回合里就等于“白嫖”了 4 个字的生成速度, 这就是推测解码能够把吞吐量提升 2 倍到 4 倍的根本原因。